

KLE Technological University
School of Electronics and Communication Engineering

Automotive Electronics Laboratory Journal

Course Code: 25EECP311

6th
Semester

2025-2026

Name: Varun Vivek Pai

SRN:01FE23BEC126

Division: C

Roll No :332

Even Sem

Automotive Electronics Lab 2025-26

Experiment No: 1

1.1 Title:

Modeling Vehicle Motion on a Flat Surface During Hard Acceleration, Deceleration, and Steady Acceleration

1.2 Aim of the Experiment:

To develop a Simulink model to simulate the longitudinal motion of a vehicle on a flat road under different driving conditions such as steady acceleration, hard acceleration, and deceleration, and to analyze its velocity and displacement response over time.

1.3 List of Equipment/Components Required:

S.No	Equipment / Component	Specification / Details
1	Computer System	Minimum 8 GB RAM, Intel i5 or higher
2	MATLAB Software	MATLAB R2022a or later
3	Simulink Toolbox	Required for dynamic system modeling
4	Vehicle Mass Parameter	1000–1500 kg (typical passenger vehicle)
5	Initial Velocity	0 m/s
6	Road Condition	Flat surface (zero slope)
7	Input Signal Blocks	Step / Signal Builder (for acceleration profiles)
8	Gain Block	Used for 1/m transformation
9	Integrator Blocks	For velocity and position calculation
10	Scope Block	For observing velocity and displacement outputs

1.4 Theoretical Background:

Vehicle longitudinal motion is governed by Newton's Second Law:

$$F = m \cdot a$$

Where:

- (F) = Net force applied on vehicle (N)
- (m) = Mass of vehicle (kg)
- (a) = Acceleration (m/s²)

Acceleration is defined as:

$$a = \frac{dv}{dt}$$

Velocity is:

$$v = \int a \, dt$$

And displacement is:

$$x = \int v \, dt$$

Motion Cases:

- **Steady acceleration:** Constant positive force applied
- **Hard acceleration:** High magnitude positive force

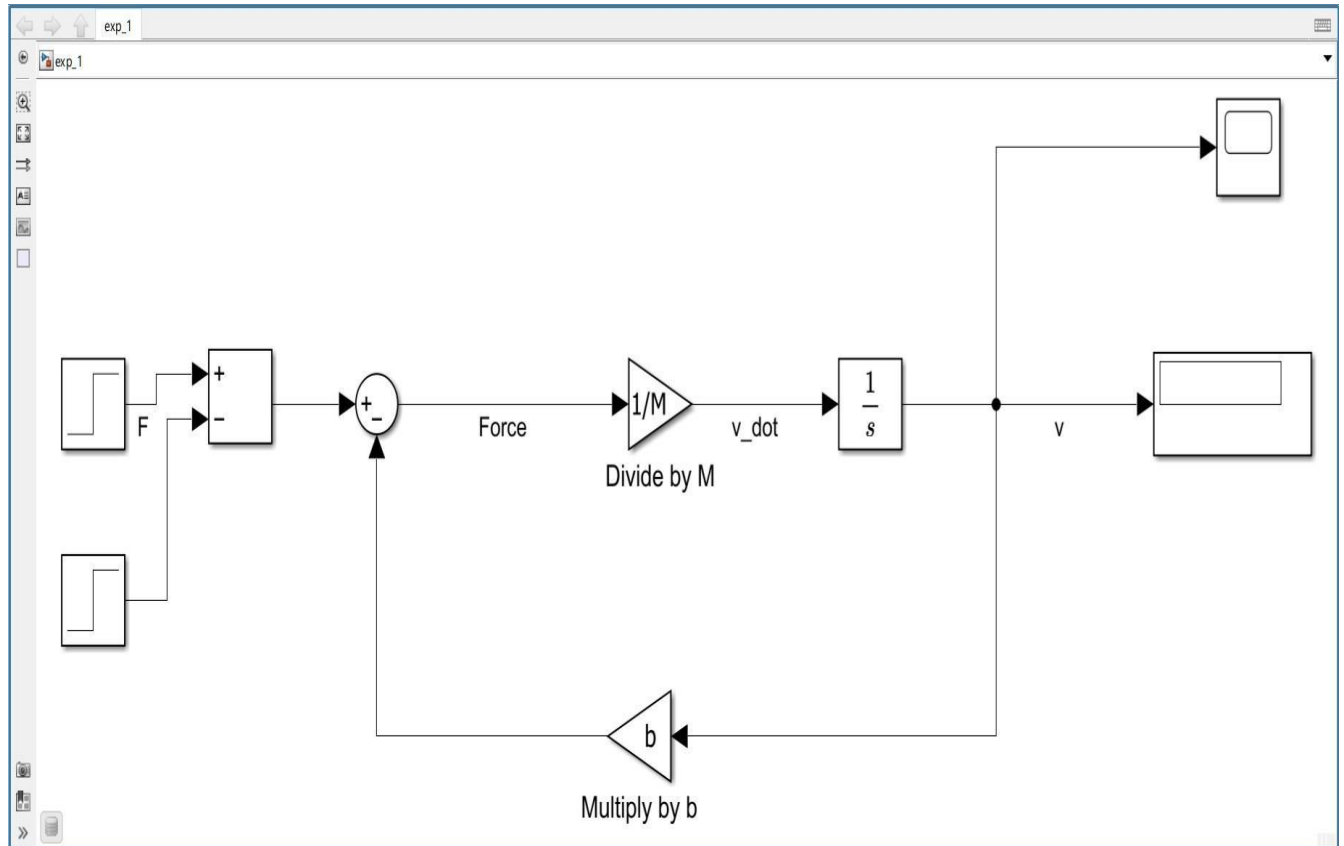
- **Deceleration:** Negative force (braking)

In a flat road condition, no gravitational component along motion exists, so only applied forces and resistances determine motion.

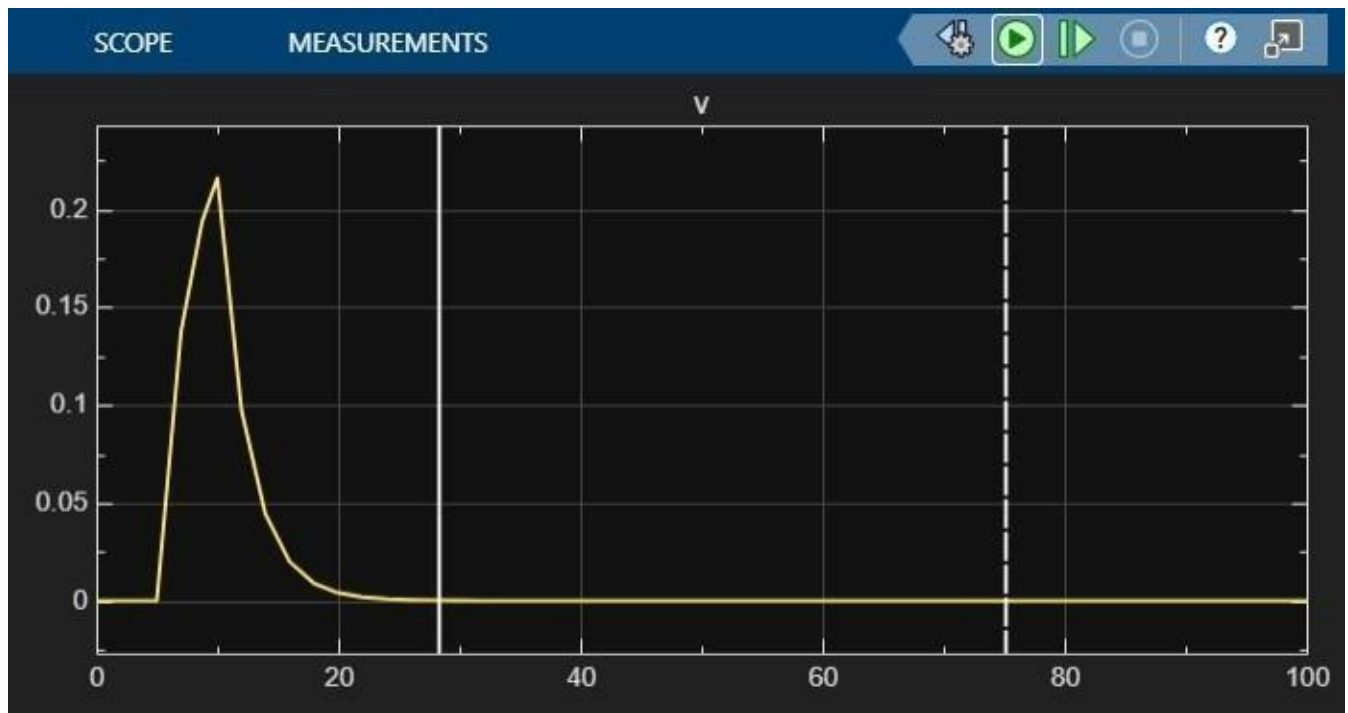
1.5 Step-by-Step Procedure:

1. Open MATLAB and launch Simulink.
2. Create a new blank model.
3. Add the following blocks:
 - Step input (for acceleration profile)
 - Gain block (1/mass) ○ Integrator (for velocity) ○ Integrator (for position) ○ Scope (to observe outputs)
4. Connect blocks as:
Force → Gain (1/m) → Acceleration → Integrator → Velocity → Integrator → Position
5. Set vehicle mass (e.g., 1200 kg).
6. Configure input signal:
 - Case 1: Steady acceleration (constant step) ○ Case 2: Hard acceleration (high step value)
 - Case 3: Deceleration (negative step value or braking input)
7. Run simulation for 20–30 seconds.
8. Observe velocity and displacement outputs in Scope.
9. Save results and plots.

1.6 Simulink Model / Code:



1.7 Results with Graphs:



Expected Outputs:

- Velocity increases linearly during acceleration
- Position increases quadratically
- During deceleration, velocity reduces gradually to zero

Graph Description:

- Velocity vs Time → straight line (positive slope for acceleration)
- Position vs Time → parabolic curve
- Deceleration phase → downward slope in velocity curve

1.8 Observations:

- Vehicle velocity increases proportionally with applied force.
- Hard acceleration results in steeper velocity curve.
- During deceleration, velocity decreases until it reaches zero.
- Displacement follows a quadratic trend under constant acceleration.
- System behaves according to Newton's laws of motion.

1.9 Conclusion:

The Simulink model successfully demonstrates the longitudinal dynamics of a vehicle under different driving conditions on a flat surface. The simulation confirms that velocity and displacement behavior follows classical mechanics, where acceleration directly depends on applied force and vehicle mass. The model helps in understanding real-world vehicle response during acceleration and braking conditions.

Experiment No: 2

2.1 Title:

Modeling of Electrical and Mechanical Subsystem

2.2 Aim of the Experiment:

To develop a Simulink model representing the interaction between electrical and mechanical subsystems and analyze the system response based on input voltage/force and resulting mechanical motion.

2.3 List of Equipment / Components Required

S.No	Equipment / Component	Specification / Details
1	Computer System	Minimum 8 GB RAM, Intel i5 or higher
2	MATLAB Software	MATLAB R2022a or later
3	Simulink Toolbox	Required for system modeling
4	Electrical Subsystem Blocks	Voltage source, resistor, inductor, gain blocks
5	Mechanical Subsystem Blocks	Mass, damper, spring (if used)
6	Input Signal	Step / sine voltage or force input
7	Integrator Blocks	For velocity and displacement
8	Scope Block	To observe system response

2.4 Theoretical Background:

Electrical and mechanical systems can be modeled using analogous variables.

Electrical System (RLC type analogy):

- Voltage (V)
- Current (I)
- Resistance (R)
- Inductance (L) Governing equation:

$$V = L \frac{dI}{dt} + RI$$

Mechanical System:

- Force (F)
- Velocity (v)
- Displacement (x)
- Mass (m)
- Damping coefficient (b)
- Spring constant (k) Governing equation:

$$F = m \frac{dv}{dt} + bv + kx$$

Electrical–Mechanical Analogy:

Electrical	Mechanical
Voltage (V)	Force (F)
Current (I)	Velocity (v)

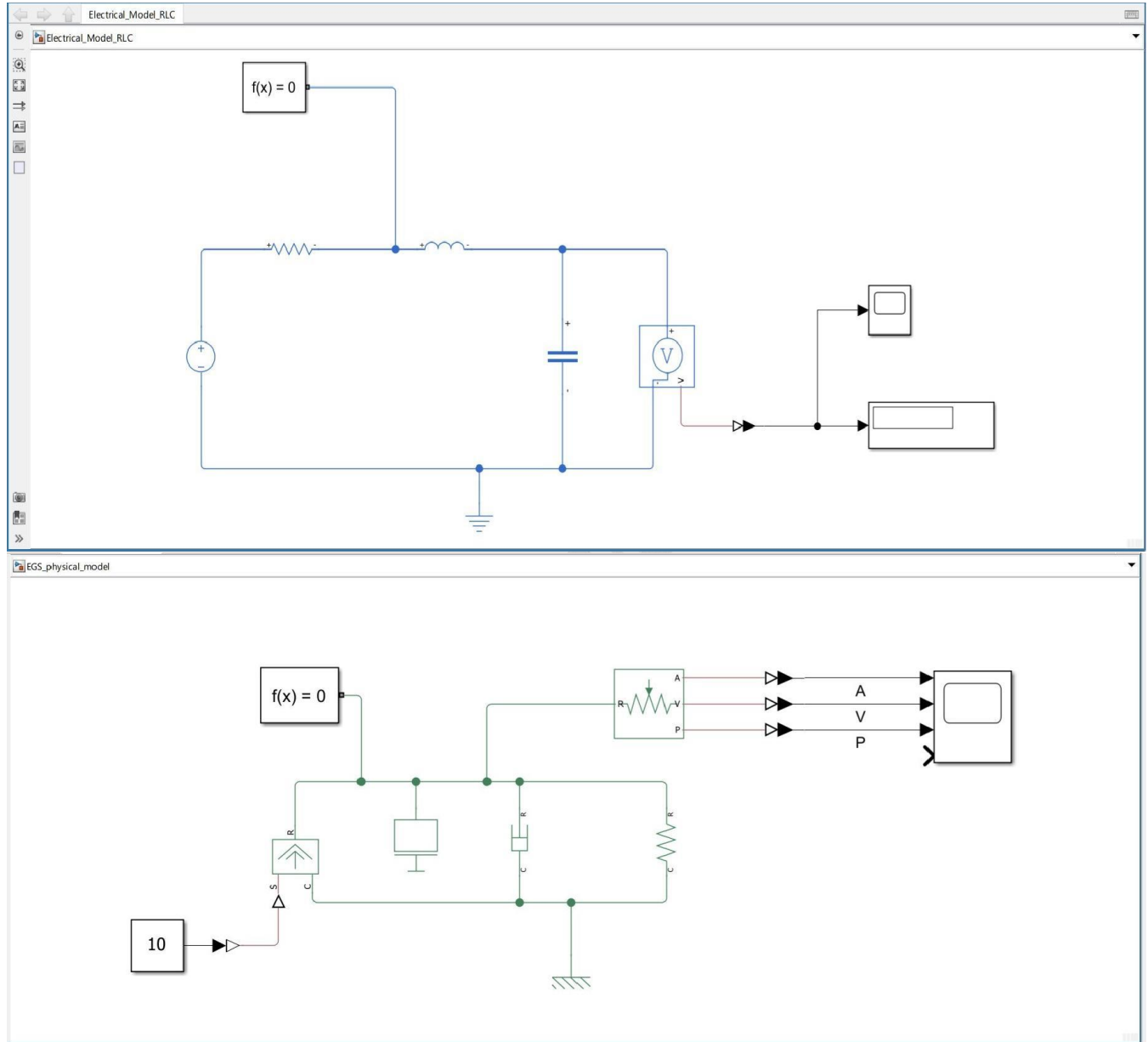
Inductance (L)	Mass (m)
Resistance (R)	Damping (b)

This analogy helps in converting electrical systems into mechanical models and vice versa.

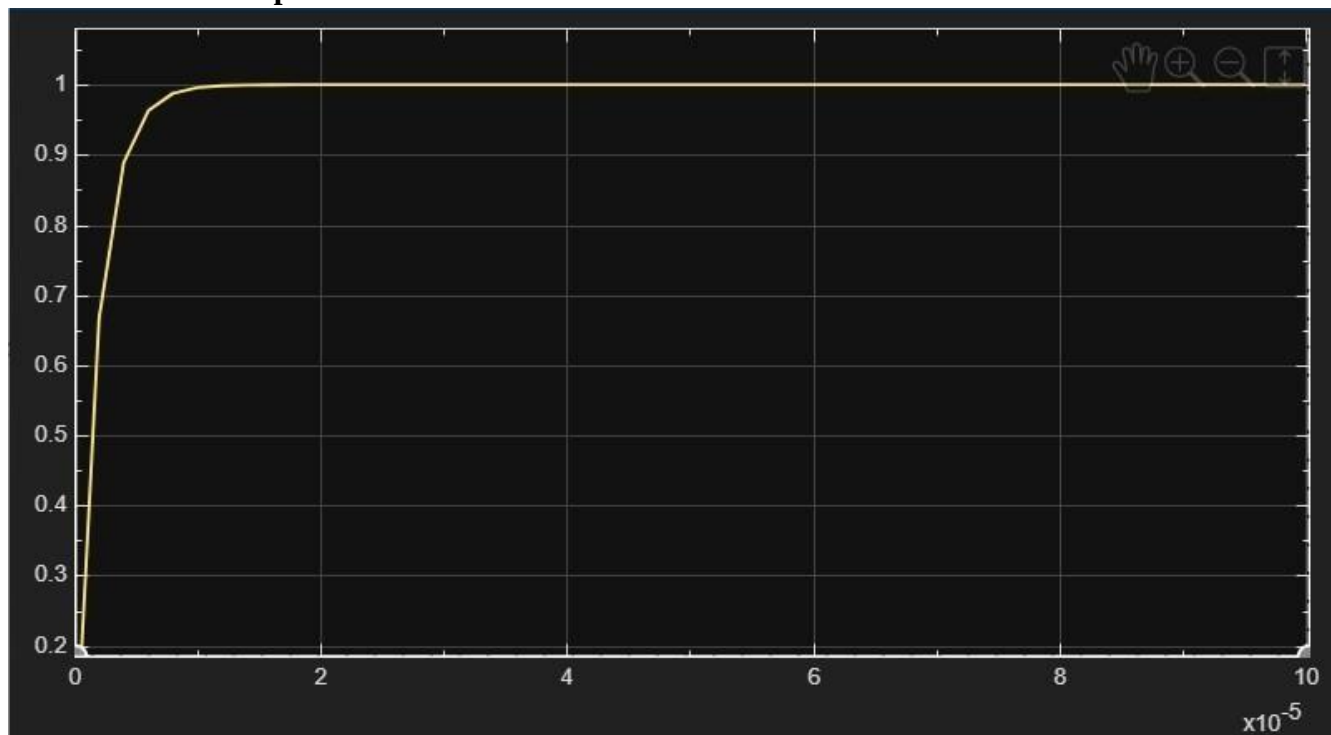
2.5 Step-by-Step Procedure:

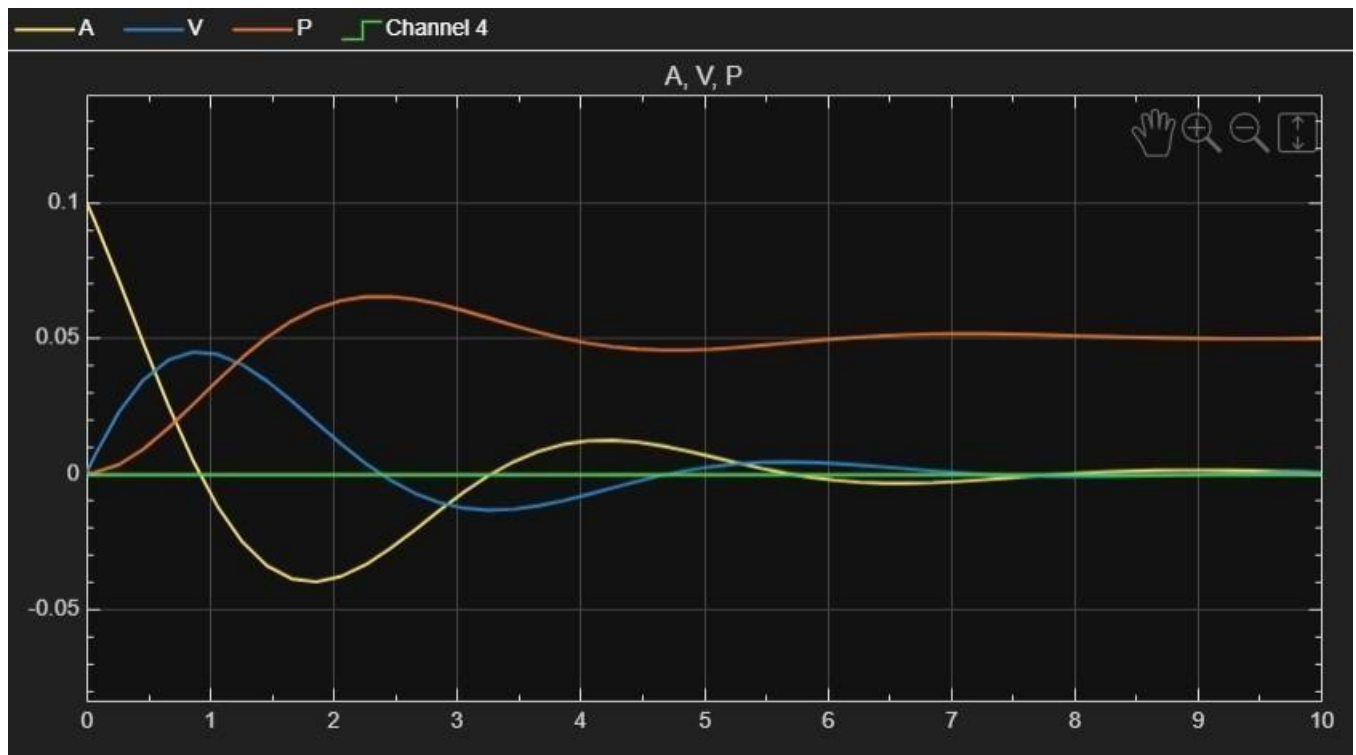
1. Open MATLAB and launch Simulink.
2. Create a new model.
3. Build electrical subsystem:
 - Add voltage source (Step input)
 - Add gain blocks (R, L parameters)
 - Connect integrator for current calculation
4. Build mechanical subsystem:
 - Add mass block (1/m)
 - Add damping gain
 - Add integrator blocks for velocity and displacement
5. Connect electrical output to mechanical input using analogy (Force ↔ Voltage).
6. Add Scope block to observe outputs.
7. Set system parameters:
 - Resistance (R)
 - Inductance (L)
 - Mass (m)
 - Damping (b)
8. Run simulation for 20–30 seconds.
9. Observe system response.

2.6 Simulink Model / Code:



2.7 Results with Graphs:





Expected Outputs:

- Electrical current increases exponentially (RL behavior)
- Mechanical velocity increases with applied force
- Displacement increases quadratically
- System shows transient response before reaching steady state

Graph Behavior:

- Current vs Time $\rightarrow$ exponential rise
- Velocity vs Time $\rightarrow$ linear increase $\square$ Position vs Time $\rightarrow$ parabolic curve

2.8 Observations:

- Electrical subsystem shows transient behavior due to inductance.
- Mechanical subsystem responds based on equivalent force input.
- Damping reduces oscillations in mechanical motion.

- Electrical and mechanical systems follow analogous behavior.
- System stability depends on R, L, m, and b values.

2.9 Conclusion:

The Simulink model successfully demonstrates the analogy between electrical and mechanical systems. The results show that electrical parameters such as resistance and inductance have equivalent mechanical effects such as damping and mass. The system behavior confirms the validity of electro-mechanical modeling techniques used in control and automotive systems.

School of Electronics Engineering

Experiment No: 3

Title:

EGAS (Electronic Gas Pedal System) Modeling and Simulation Using Simulink and Realization on Hardware Platform

3.1 Aim of the Experiment:

- To explain the functioning of sensors and actuators in the Electronic Gas Pedal System (EGAS).
- To model and simulate sensors, actuators, and control algorithms using Simulink. To implement and verify the system response using hardware realization.

3.2 List of Equipment / Components Required

Table 3.1: List of Hardware and Software used to conduct the experiment

Sl. No	Hardware / Software	Specifications
1	Integrator Block	Used for system dynamics
2	Gain Block	Gain value = 40
3	Subtractor (Sum Block)	Used for error calculation
4	Step Input Block	Input range: 0 to 1
5	MATLAB Software	R2022a or later
6	Simulink Toolbox	For modeling and simulation

7	Hardware Interface	Arduino / dSPACE / ECU setup (if available)
8	Sensor System	Accelerator pedal position sensor

3.3 Theoretical Background:

The **Electronic Gas Pedal System (EGAS)** is a drive-by-wire technology where the mechanical linkage between the accelerator pedal and throttle body is replaced by electronic sensors and actuators.

Working Principle:

- Pedal position is measured using a position sensor.
- The signal is sent to the Electronic Control Unit (ECU).
- ECU computes the error between desired and actual throttle position.
- Control algorithm generates appropriate control signal. □ Actuator adjusts throttle opening accordingly.

Control Equation:

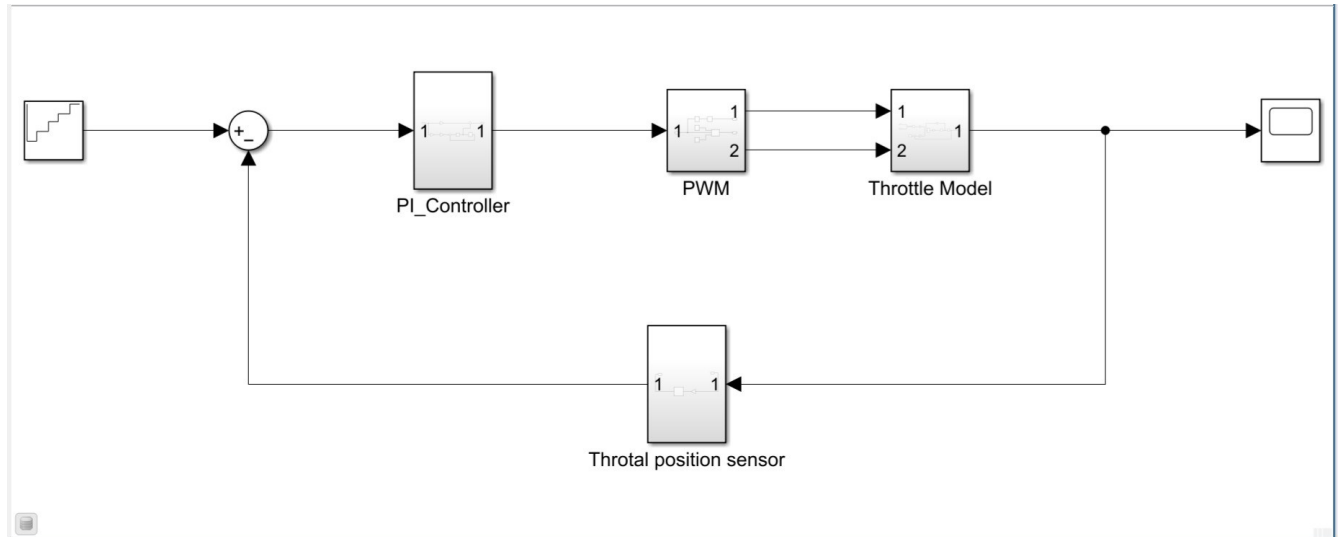
$$e(t) = r(t) - y(t)$$

Where:

- $r(t)$ = desired pedal input
- $y(t)$ = actual throttle response
- $e(t)$ = error signal

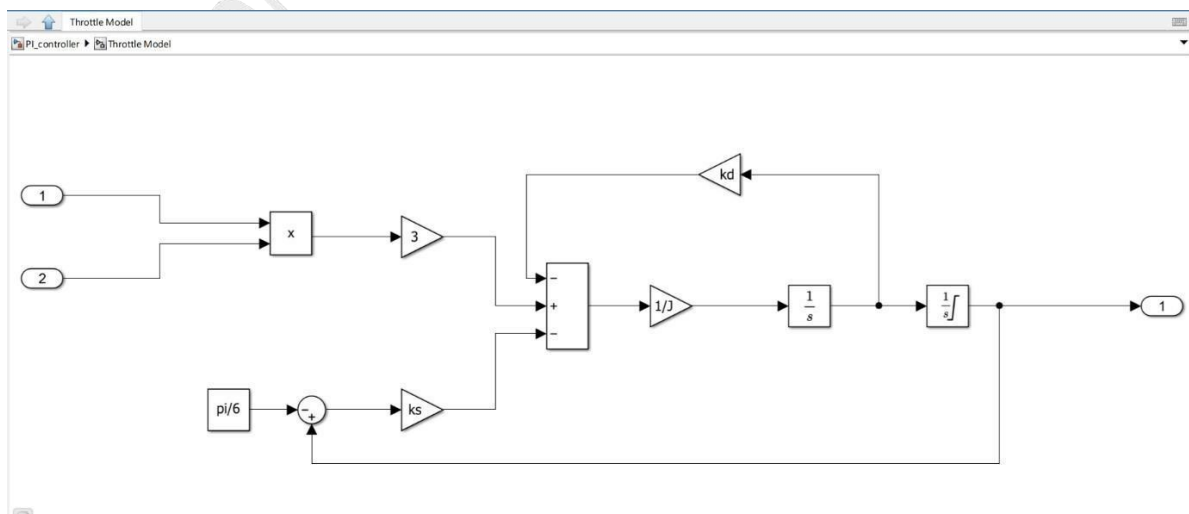
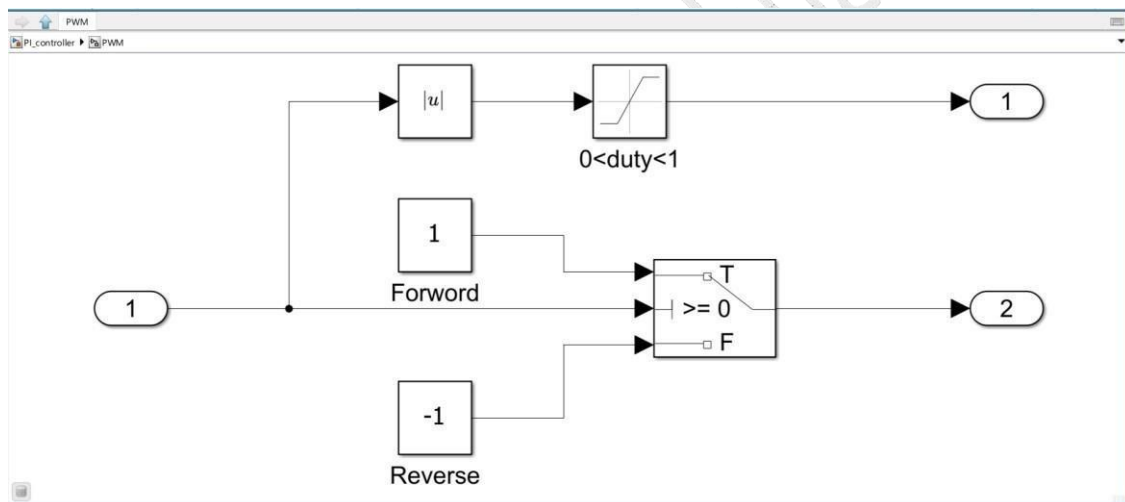
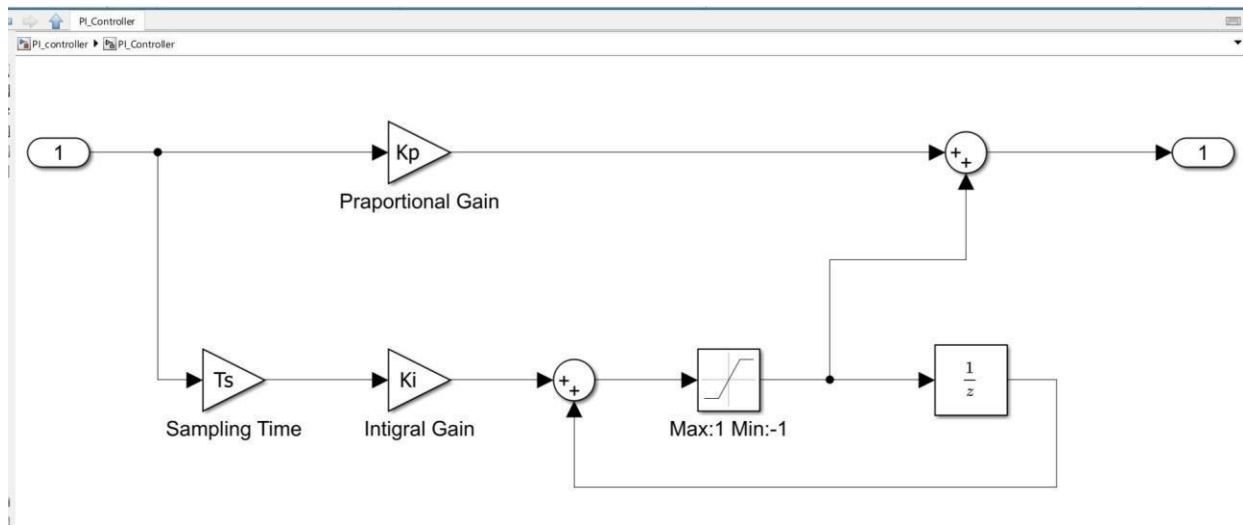
The system operates as a **closed-loop feedback control system** ensuring accurate throttle response.

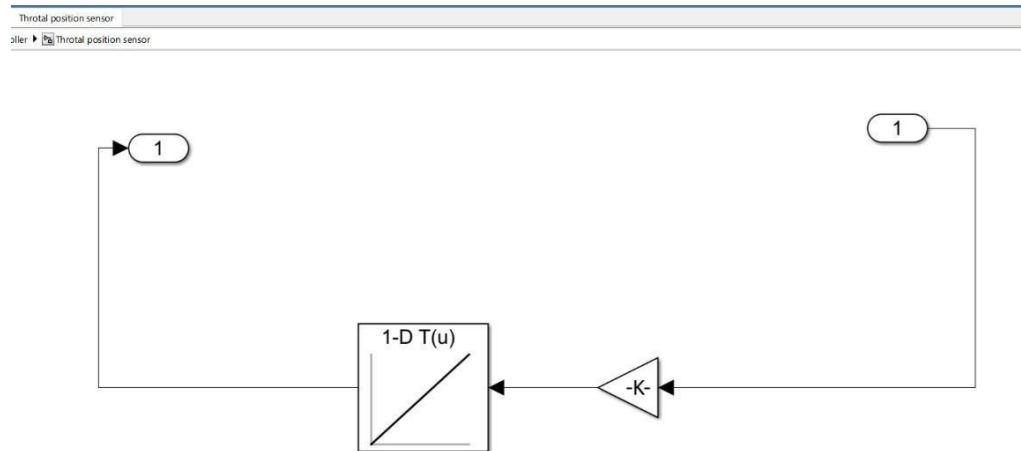
3.4 Simulink Model / Code:



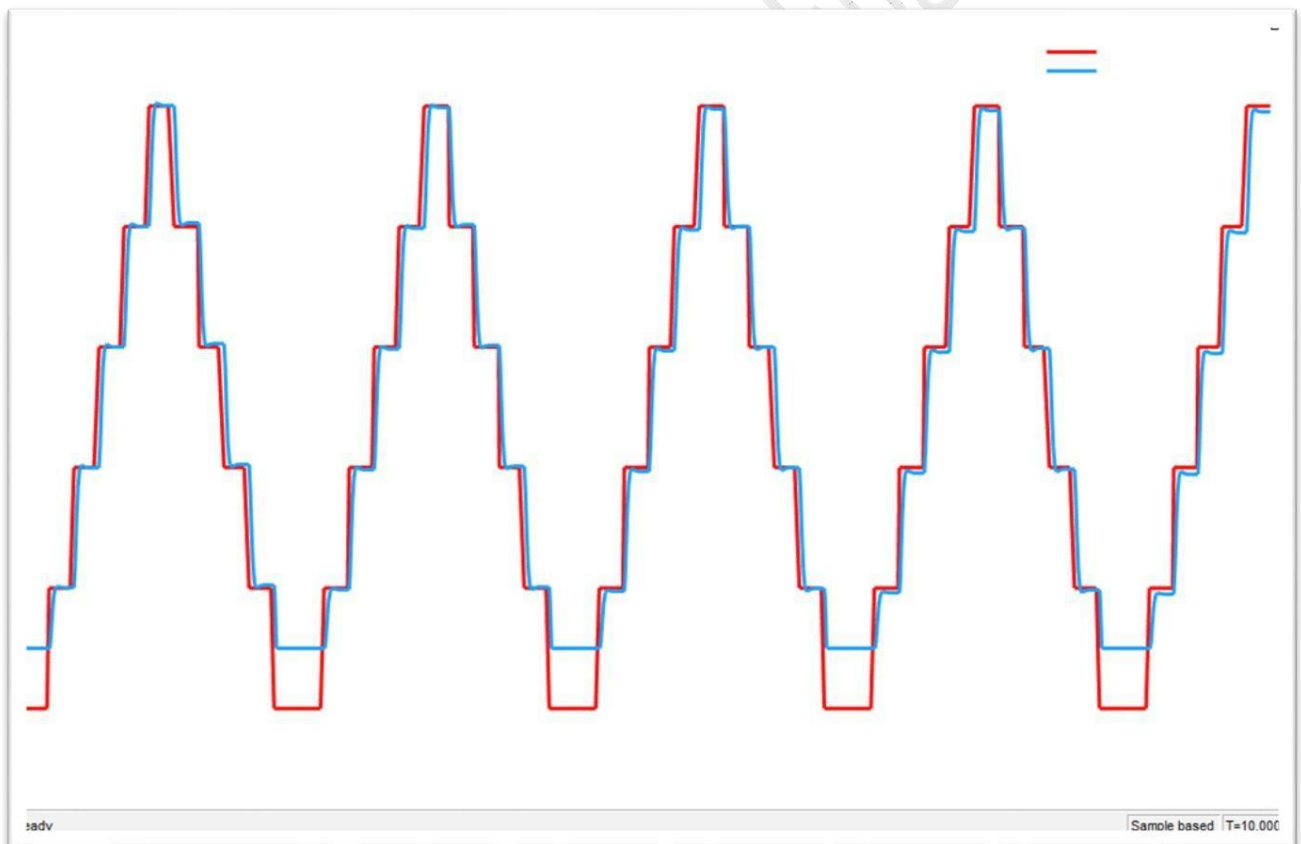
Automotive Electronics Laboratory Journal

25EECP311





3.5 Results with Graphs:



Expected Output:

- Step input represents sudden pedal press.
- Output shows smooth rise due to system dynamics.
- System reaches steady-state after transient response.

Graph Behavior:

- Input: Step ($0 \rightarrow 1$)
- Output: Smooth exponential-like curve
- Error: Gradually decreases to zero

3.6 Observations:

- EGAS system responds smoothly to step input.
- Feedback control improves accuracy and stability.
- Gain value significantly affects response speed.
- Integrator introduces realistic actuator dynamics.
- Hardware realization shows similar behavior to simulation with minor delays.

3.7 Conclusion:

The EGAS (Electronic Gas Pedal System) was successfully modeled and simulated using Simulink, and its behavior was validated through hardware realization. The results confirm that the system operates effectively as a closed-loop feedback control system, providing accurate and stable throttle response in automotive applications.

Experiment No: 4

Title:

Modeling and Simulation of Fuel Control System (MAF Based) on Simulink Platform

4.1 Aim of the Experiment:

- To model and simulate a Fuel Control System using Simulink.
- To understand the role of Mass Air Flow (MAF) based fuel calculation in engine control. □
To analyze fuel delivery behavior under different engine operating conditions.

4.2 List of Equipment / Components Required

Table 4.1: List of Hardware and Software used to conduct the experiment

Sl. No	Hardware / Software	Specifications
1	Switch Block	Used for mode selection (conditions ON/OFF)
2	Gain Block	Used for fuel scaling / air-fuel ratio control
3	Constant Block	Provides fixed input values (air flow / lambda)
4	1D Lookup Table	Maps MAF sensor input to fuel injection quantity
5	MATLAB Software	R2022a or later
6	Simulink Toolbox	For modeling and simulation

4.3 Theoretical Background:

A **Fuel Control System** regulates the amount of fuel injected into the engine based on air intake to maintain an optimal air–fuel ratio.

MAF-Based Fuel Control:

The **Mass Air Flow (MAF) sensor** measures the amount of air entering the engine. The ECU uses this value to calculate required fuel injection.

Basic Relationship:

$$\text{Fuel Quantity} \propto \frac{\text{Air Mass}}{\text{Air Fuel Ratio}}$$

Air–Fuel Ratio (Stoichiometric):

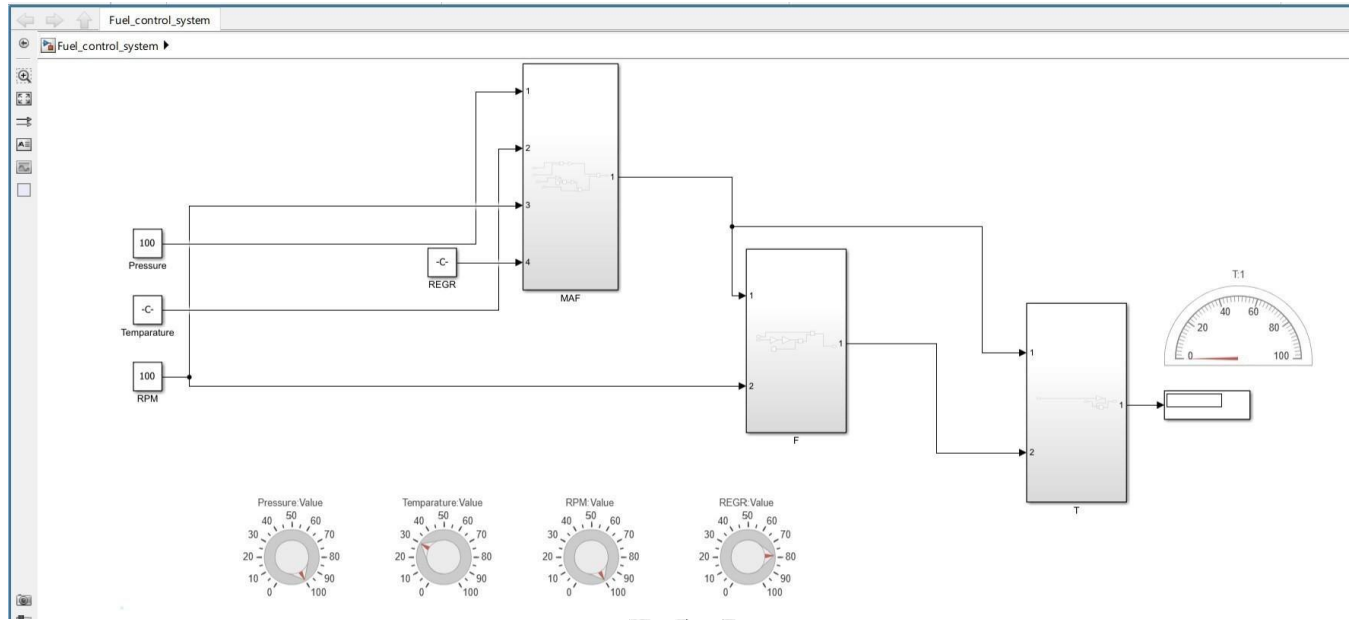
For petrol engines: AFR
approximately 14.7:1

This means:

- 14.7 parts air → 1 part fuel
- **System Function:**
 - MAF sensor reads incoming air
 - ECU calculates required fuel
 - Fuel injector delivers correct fuel quantity
 - Feedback ensures efficient combustion

4.4 Simulink Model / Code:

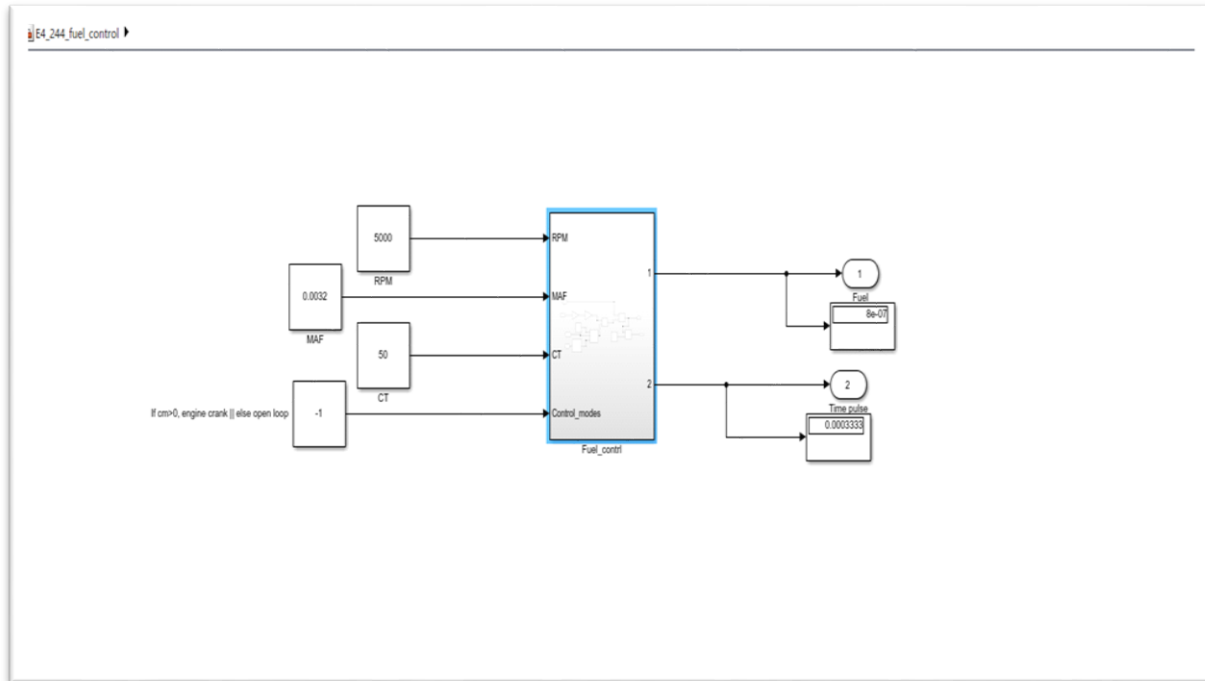
Block Diagram:



Explanation:

- **MAF sensor input** represents air entering engine.
- **Lookup table** converts air flow to fuel requirement.
- **Gain block** adjusts fuel based on AFR ratio.
- **Switch block** selects operating conditions (idle / acceleration). □ Output represents final fuel injection signal.

4.5 Results with Graphs:



Expected Response:

- Fuel injection increases with increase in air flow.
- Lookup table creates nonlinear response behavior.
- Switch block causes step changes under different operating conditions.

Graph Behavior:

- MAF vs Fuel: nonlinear increasing curve
- Step input response: sudden rise in fuel delivery
- Steady-state region: stable fuel output for constant air flow

4.6 Observations:

- Fuel injection depends directly on MAF sensor input.
- Lookup table improves accuracy of real engine behavior.
- System ensures proper air-fuel mixture for combustion efficiency.

- Switch block allows different engine operating modes. □ Output remains stable under constant input conditions.

4.7 Conclusion:

The Fuel Control System using MAF sensor was successfully modeled and simulated in Simulink. The results show that fuel injection varies according to air intake and maintains an optimal air–fuel ratio. This confirms the effectiveness of MAF-based fuel control strategies in modern engine management systems.

School of Electronics Engineering

Experiment No: 5

Title: Modeling seat belt warning system, and vehicle speed control based on the gear input using state flow and realization on the hardware platform.

5.1 Aim of the experiment:

- Understand the sensors and actuator working principles.
- Model and simulate the sensors, actuators and control algorithm.
- Generate C-code from the model to deploy onto the target hardware

5.2 List of Equipment/ Components required to conduct the experiment with specifications

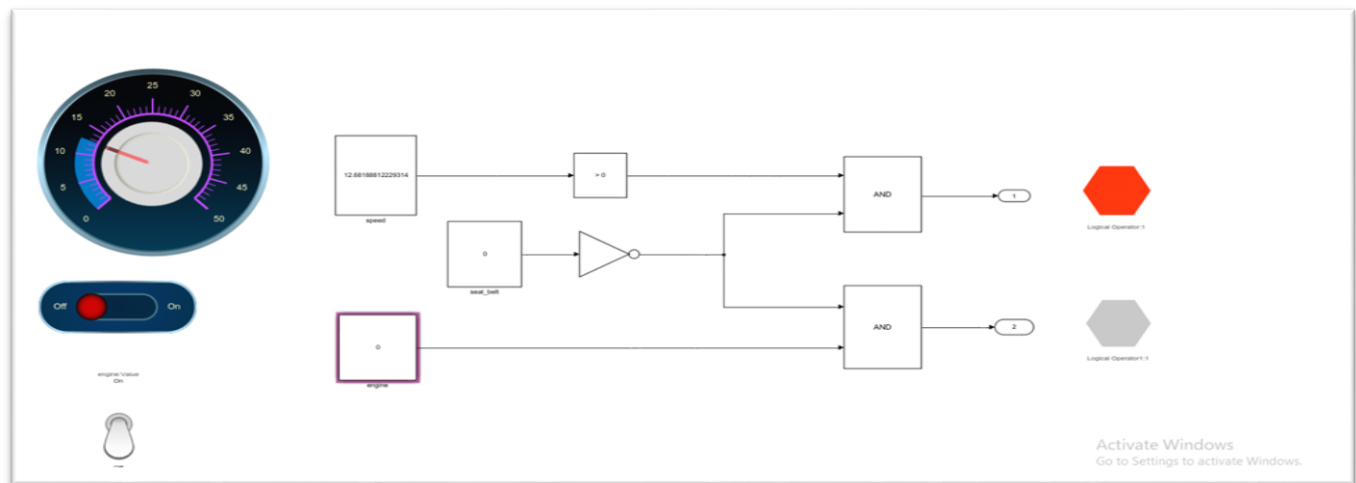
Table 5.1: List of all Hardware and Software used to conduct the experiment

Sl. No	Hardware/Software	Specifications
1	Cortex M3 mbed	Null
2	buzzer	-
3	leds	-
4	Potentiometer	-

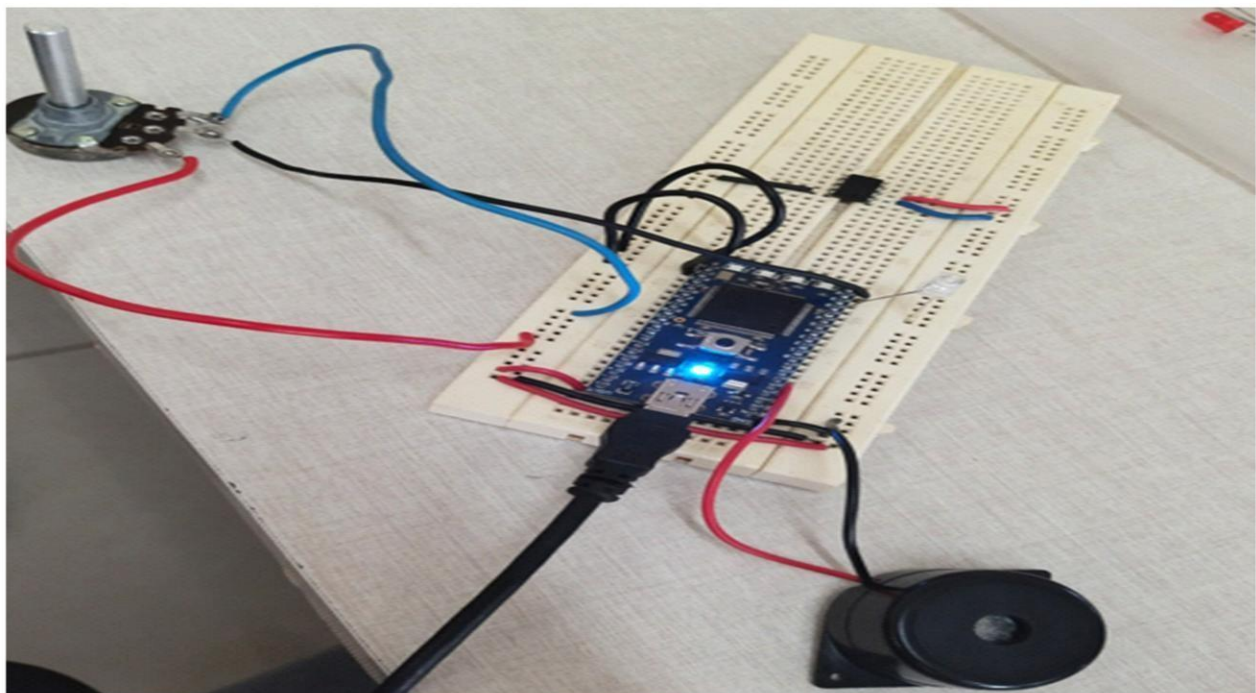
5.3 Theoretical background for the experiment:

Stateflow in Simulink enables the modeling of event-driven systems such as seat belt warning and vehicle speed control, incorporating sensor inputs and actuator outputs. This approach allows for simulation, control algorithm design, and C-code generation for deployment on hardware, ensuring accurate and efficient system behavior.

5.4 Simulink model/code:



5.5 Implementation on hardware and Code:



```
#include "mbed.h"
```

```
AnalogIn vehicleSpeed(p20);  
  
DigitalOut buzz(p7);  
  
DigitalOut led(p19);  
  
DigitalIn engine_status(p21);  
  
DigitalIn seat_belt_status(p22);  
  
#define BLINKING_RATE1 50ms
```

```
int main() {  
    while(1) {  
        led = 0;  
        buzz = 0;  
  
        if (vehicleSpeed > 0.0f && seat_belt_status == 0) {  
            led = 1;  
            buzz = 1;  
        }  
        else if (vehicleSpeed == 0.0f && engine_status == 1) {  
            led = 1;  
            buzz = 0;  
        }  
        else if (vehicleSpeed > 2.0f) {
```

```
        buzz = 1;  
    }  
    ThisThread::sleep_for(BLINKING_RATE1);  
}  
}
```

5.6 Conclusion:

I have verified the output using Simulink tool and on hardware.

School of Electronics Engineering

Experiment No: 6

Title: Modeling and simulation of wiper control system using state flow and realization on the hardware platform.

6.1 Aim of the experiment:

- Understand the sensors and actuator working principles.
- Model and simulate the sensors, actuators and control algorithm.
- Generate C-code from the model to deploy onto the target hardware

6.2 List of Equipment/ Components required to conduct the experiment with specifications

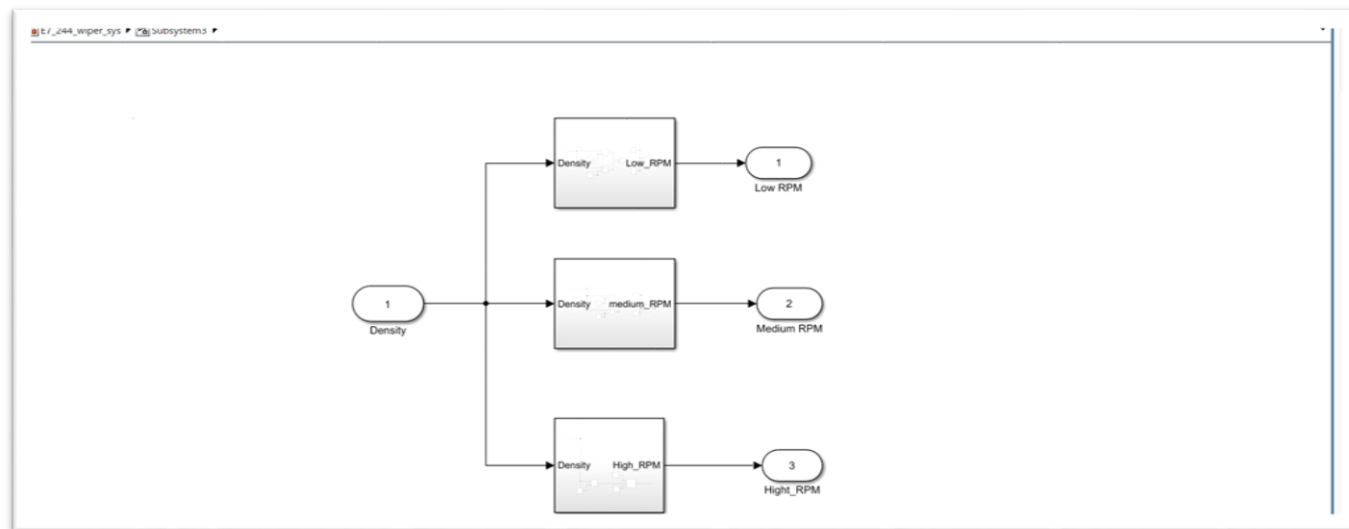
Table 6.1: List of all Hardware and Software used to conduct the experiment

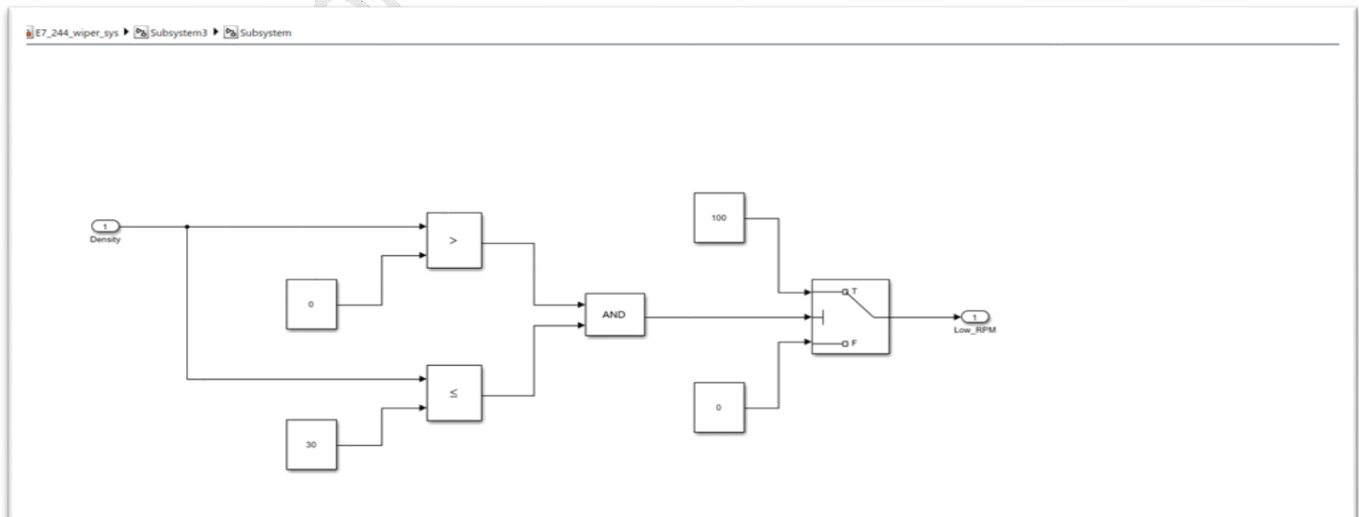
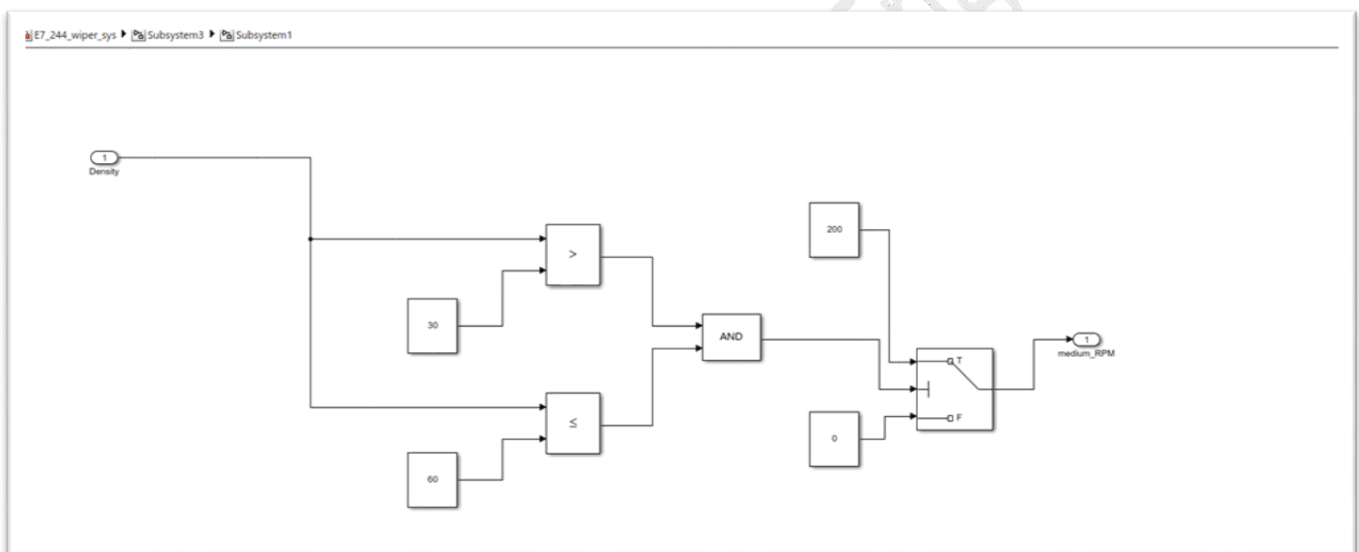
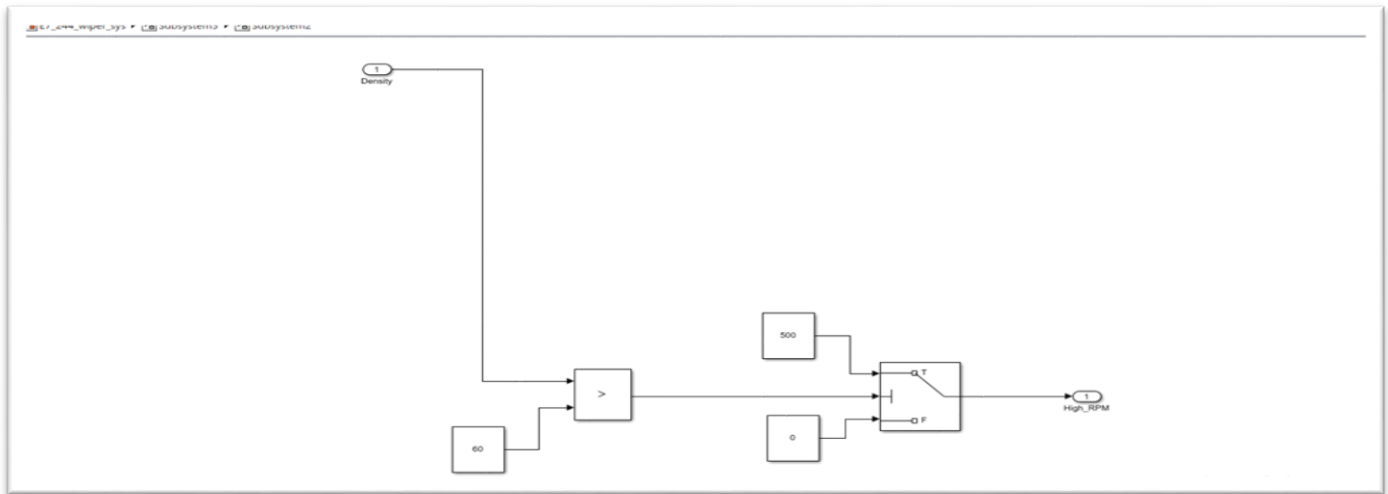
Sl. No	Hardware/Software	Specifications
1	Cortex M3 mbed	Null
2	buzzer	-
3	leds	-
4	Potentiometer	-

6.3 Theoretical background for the experiment:

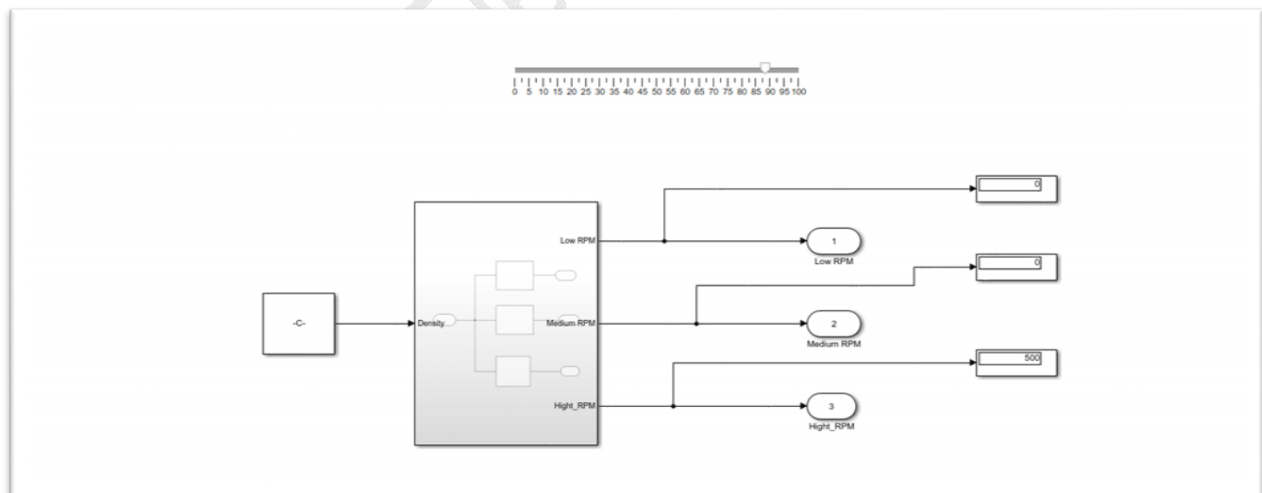
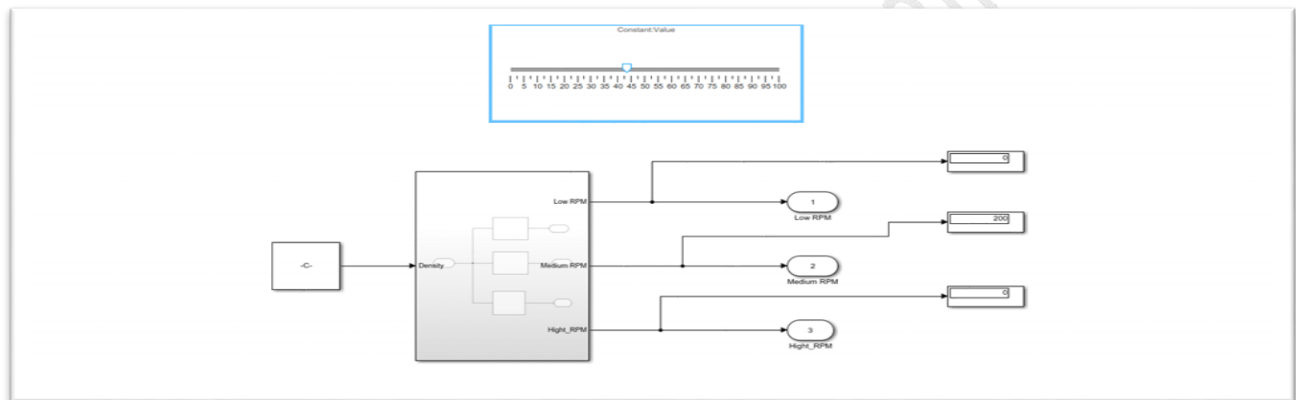
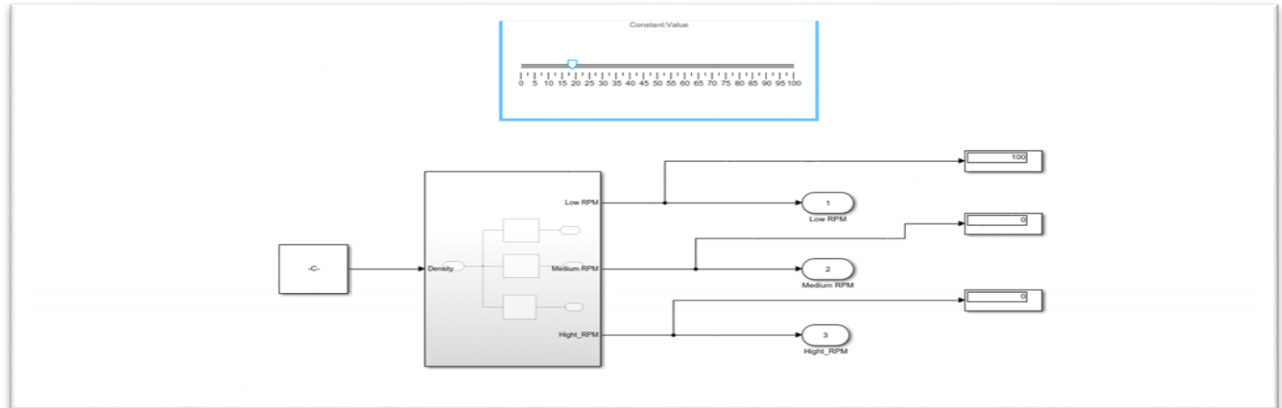
Stateflow in Simulink facilitates the modeling of a wiper control system by integrating sensor inputs and actuator outputs to simulate and optimize the control algorithm. This process includes generating C-code from the model for deployment on hardware, ensuring precise and reliable operation of the wiper system.

6.4 Simulink Model:





Results:



6.5 Code:

```
#include "mbed.h"
```

```
AnalogIn RainfallSpeed(p20);
DigitalOut led1(LED1);
DigitalOut led2(LED2);
DigitalOut led3(LED3);
DigitalIn engine_status(p21);
DigitalIn wiper_status(p22); // main()
runs in its own thread in the OS int
main() {
    while (true)
    {
        if(engine_status==1 && wiper_status==1)
        {
            if(RainfallSpeed>=0.1f && RainfallSpeed<0.2f)
            {
                led1=1;
                led2=0;
                led3=0;
            }
            else if(RainfallSpeed>=0.2f && RainfallSpeed<0.3f)
            {
                led1=0;
                led2=1;
                led3=0;
            }
            else if(RainfallSpeed>=0.3f)
            {
                led1=0;
                led2=0;
                led3=1;
            }
        }
    }
}
```

```
    }  
  } else  
  {  
    led1=0;  
    led2=0;  
    led3=0;  
  }  
}  
}
```

6.7 Conclusion:

I have verified the model in Simulink and on hardware also.

Experiment No: 7

Title: Modeling and simulation of the power window system of an automobile and realization on the hardware platform.

7.1 Aim of the experiment:

- Model and simulate the opening and closing windows and sunroof/Adjusting mirrors and headlights/Locking and unlocking doors functionality.
- Model and simulate the functionalities using state machines

7.2 List of Equipment/ Components required to conduct the experiment with specifications

Table 7.1: List of all Hardware and Software used to conduct the experiment

Sl. No	Hardware/Software	Specifications
1	Cortex M3 mbed	Null
2	Wires	-
3	leds	-
4	Potentiometer	-

7.3 Theoretical background for the experiment:

Using Simulink's Stateflow, the power window system of an automobile can be modeled to simulate the operation of windows, sunroofs, mirrors, headlights, and door locks. This approach leverages state machines to design and optimize control algorithms, facilitating seamless hardware implementation for reliable functionality.

7.4 Implementation Code:

```
#include "mbed.h"
```

```
// main() runs in its own thread in the OS
```

```
DigitalIn pb1(p8); // To move up
```

```
DigitalIn pb2(p9); // To move down
```

```
DigitalIn pb3(p20); // object
```

```
DigitalOut led1(LED1);//Up of window
DigitalOut led2(LED2);//Down of the window
#define delay1 400ms
#define delay2 500ms
#define delay3 100ms
```

```
int old_pb1=1,old_pb2=1;
int new_pb1,var1=0,new_pb2,var2=0,c;
```

```
int main() {
    pb1.mode(PullUp);
    pb2.mode(PullUp);
    pb3.mode(PullUp);
    while (true)
    {
        new_pb1=pb1;
        new_pb2=pb2;    if(new_pb1==0
        && old_pb1==1)
        {
            if(var1==0)
            {
                c=0;
                var1=1;
                var2=0;
                led1=1;
                while(c<4)
                {
                    if(pb3==0)
                    {
                        var2=1;
                        var1=0;
                        led1=0;
                        led2=1;
```

```
                ThisThread::sleep_for(delay3);
                ThisThread::sleep_for(delay3);
```

```
        led2=0;
break;    }
else      {
        c+=1;
        ThisThread::sleep_for(delay3);    }    }
    led1=0;
    }
}
old_pb1=new_pb1;
if(new_pb2==0 && old_pb2==1)
{
    if(var2==0)
    {
var2=1;
var1=0;
led2=1;
        ThisThread::sleep_for(delay1);
led2=0;
    }
}

    old_pb2=new_pb2;
}
}
```

7.5 Conclusion:

I have verified the ckt on hardware and dumped the code in hardware results are obtained correctly.

Experiment No: 8

Title:

Model driven integration using CAN communication for Vehicle speed control based on the gear input.

8.1 Aim of the experiment:

- Establish CAN network using ARM Cortex M3.
- Explore automotive protocol tools such as CANalyzer/Busmaster to analyze the CAN signals

8.2 List of Equipment/ Components required to conduct the experiment with specifications

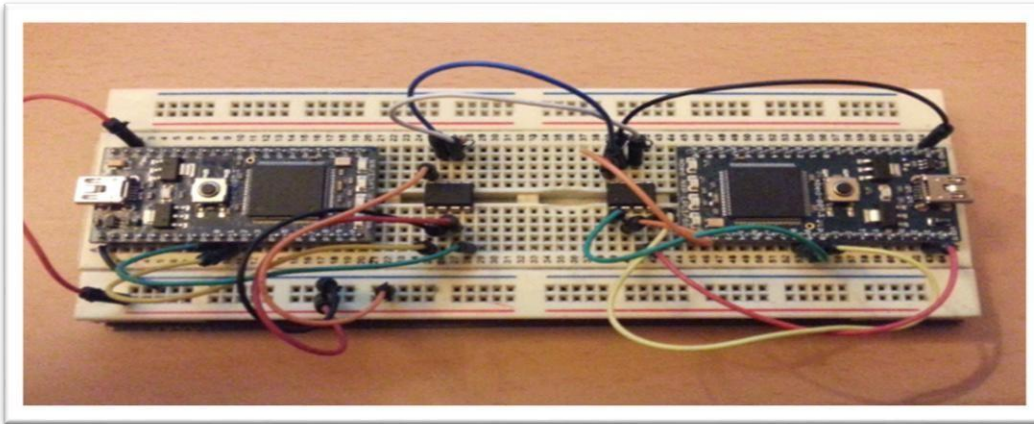
Table 8.1: List of all Hardware and Software used to conduct the experiment

Sl. No	Hardware/Software	Specifications
1	Cortex M3 mbed	Null
2	Wires	-
3	leds	-
4	Potentiometer	-

8.3 Theoretical background for the experiment:

Model-driven integration utilizes CAN communication to synchronize vehicle speed control with gear input, enhancing automotive functionality and safety. By establishing a CAN network using ARM Cortex M3 and analyzing CAN signals through tools like CANalyzer/Busmaster, precise control and monitoring of vehicle systems are achieved, ensuring efficient and reliable operation.

8.4 Implementation on hardware:



8.5 Implementation Code:

CAN Transmitter code:

```
#include "mbed.h"

//Serial pc(USBTX, USBRX);      // tx, rx for Tera Term output

DigitalOut led1(LED1);          // LED1 displays message sent Status

CAN can1(p9, p10);              // CAN interface char counter = 0;

#define BLINKING_RATE1 1000ms    // Counter Variable to store number of sent messages

int main() { printf("sending a message via CANbus... ");

    while (1) {

        // send value to CAN bus and monitor return value to check if CAN    //
        message was sent successfully. If so display, increment and toggle    if
        (can1.write(CANMessage(1, &counter, 1))) {      printf("CANBus
        Message sent: %d\n", counter);    // display message

            counter++;                                // increment message counter value

            led1 = !led1;                            // toggle LED1 to show message sent
```



```
    }else{    can1.reset();                // Reset CANbus if there is a
problem
    }

    ThisThread::sleep_for(BLINKING_RATE1);        // wait a second
}
}
```

CAN Receiver Code:

```
#include "mbed.h"

//Serial pc(USBTX, USBRX);    // tx, rx for Tera Term output

DigitalOut led2(LED2);    // status LED CAN

can1(p30, p29);    // CAN interface int

main() {

    CANMessage msg;    // create empty CAN message

    printf("received a CANbus message...\n"); // Write to serial terminal

    while(1) {    if(can1.read(msg)) {    // if message is available, read into
msg    printf("Message received: %d\n", msg.data[0]); // display message
data

        led2 = !led2;                // toggle receive status LED
    }

    }

}
```

8.6 Conclusion:

I have verified the CAN bus on hardware platform using two cortex M3 boards , both transmitter and receiver are verified.

School of Electronics Engineering

Experiment 9 (Open Ended experiment)

9.1 Title of the Experiment

Modeling and simulation of a throttle-by-wire (TBW) system using MATLAB/Simulink.

9.2 Aim of the Experiment

To model and simulate a throttle-by-wire (TBW) system using MATLAB/Simulink and analyze the closed-loop control of throttle position using a PID controller.

9.3 Theoretical Background for the Experiment

Throttle-by-wire (TBW) is an electronic throttle control system that replaces the conventional mechanical linkage between the accelerator pedal and the throttle valve with an electronic control mechanism. The accelerator pedal position is measured electronically and converted into a command signal. This command signal is transmitted to the electronic control unit (ECU), which determines the required throttle opening.

In the developed model, the desired throttle position is compared with the actual throttle position to generate an error signal. The error signal is processed by a PID controller, which generates the control action required to drive the throttle actuator. The actuator adjusts the throttle opening according to the controller output. The actuator output is fed back to the controller, forming a closed-loop control system.

The feedback mechanism continuously minimizes the error between the desired and actual throttle positions, thereby improving system stability, response, and tracking accuracy. Throttle-by-wire systems are widely used in modern vehicles to enhance engine performance, fuel efficiency, and driving comfort.

The error signal is given by:

$$e(t) = r(t) - y(t)$$

where, $e(t)$ = error signal $r(t)$

= desired throttle position

$y(t)$ = actual throttle position

The output of the PID controller is given by:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d [de(t)/dt]$$

where,

K_p = proportional gain

K_i = integral gain

K_d = derivative gain

The throttle actuator is modeled using the transfer

function: $G(s) = 1/(0.5s + 1)$ where,

$G(s)$ = actuator transfer function s

= Laplace variable

The transfer function represents the dynamic behavior of the throttle actuator and determines how quickly the throttle responds to changes in the control signal.

9.4 Block Diagram

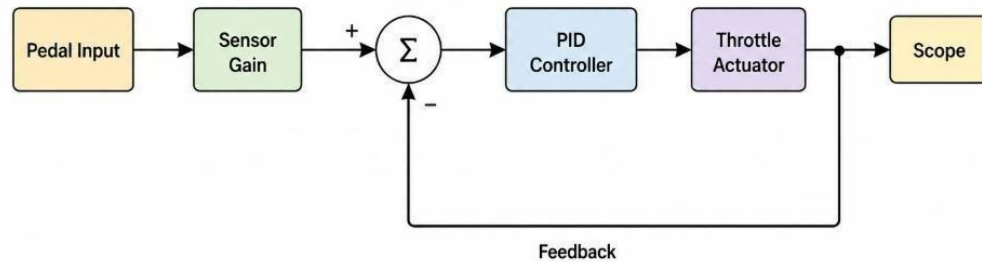


Figure 9.1 Block diagram of the throttle-by-wire system.

9.5 Procedure

1. Open MATLAB and create a new Simulink model.
2. Add the Step, Gain, Sum, PID Controller, Transfer Function, and Scope blocks.
3. Configure the Step block to apply a pedal input of 70% at 1 second.
4. Set the gain value of the sensor block to 1.
5. Configure the PID controller with proportional gain $K_p = 2$, integral gain $K_i = 1$, and derivative gain $K_d = 0.1$.
6. Configure the transfer function block with numerator equal to 1 and denominator equal to [0.5 1].
7. Connect the blocks to form a closed-loop feedback system.
8. Run the simulation.
9. Observe the response using the Scope block.

9.6 Simulink Model

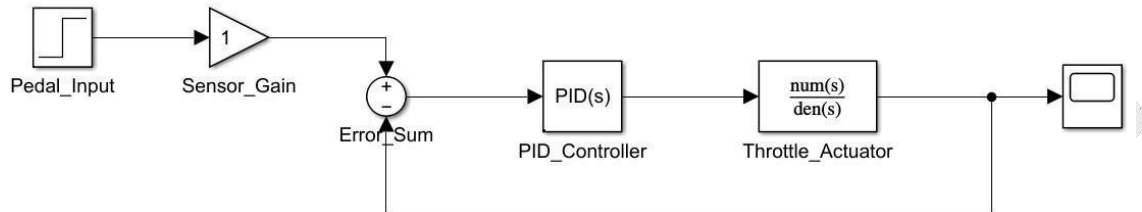


Figure 9.2 Simulink model of the throttle-by-wire system. 9.7

Simulation Results



Figure 9.3 Simulation response of the throttle-by-wire system.

The simulation result shows that the throttle position remains at zero before the pedal input is applied. At 1 second, a pedal input of 70% is introduced. The PID controller generates the required control signal, causing the throttle actuator to respond and gradually reach the desired position. The system settles at the commanded value with a stable response.

9.8 Observations

- The system operates as a closed-loop control system.
- The PID controller continuously minimizes the error between the desired and actual throttle positions.
- The throttle actuator responds smoothly to the applied pedal input.
- The feedback loop improves system stability and tracking accuracy.
- The throttle position reaches the desired value after a short transient period.

9.9 Result

A throttle-by-wire system was successfully modeled and simulated using MATLAB/Simulink. The PID controller effectively controlled the throttle actuator through a closed-loop feedback mechanism and enabled accurate tracking of the desired throttle position.

9.10 Conclusion

The throttle-by-wire system was successfully implemented and analyzed using MATLAB/Simulink. The simulation demonstrated the effectiveness of PID control in regulating throttle position. The closed-loop feedback system provided stable operation and accurate tracking of the driver's throttle command.

Experiment 10 (Open Ended Experiment)

10.1 Title of the Experiment

Modeling and simulation of an Advanced Driver Assistance System (ADAS) with Automatic Emergency Braking (AEB) using MATLAB/Simulink.

10.2 Aim of the Experiment

To model and simulate an Advanced Driver Assistance System (ADAS) with Automatic Emergency Braking (AEB) using MATLAB/Simulink and analyze the vehicle braking response when an obstacle is detected within a predefined safety distance.

10.3 Theoretical Background for the Experiment

Advanced Driver Assistance Systems (ADAS) are electronic systems designed to improve vehicle safety and driving comfort by assisting the driver in various driving situations. One of the important features of ADAS is Automatic Emergency Braking (AEB), which helps prevent collisions by automatically applying the brakes when an obstacle is detected within a critical distance.

In an AEB system, sensors such as radar, LiDAR, or cameras continuously monitor the distance between the vehicle and obstacles ahead. The measured distance is compared with a predefined safety threshold. If the detected distance becomes smaller than the threshold value, the AEB controller activates the braking system automatically.

In this experiment, the obstacle distance is represented by a ramp signal that decreases with time. A relational operator compares the measured distance with the safety threshold. When the distance falls below the threshold, the braking force is applied automatically through a switch block. The braking force generates vehicle deceleration, which reduces the vehicle speed. The speed response is observed using a scope block.

The decision logic used in the system is represented as:

Brake Command = ON, if Distance < Threshold

Brake Command = OFF, if Distance $\geq$ Threshold

The vehicle acceleration is calculated using Newton's second law:

$$F = ma$$

Therefore,

$$a = F/m$$

where, a = vehicle

acceleration (m/s^2) F =

braking force (N) m = vehicle

mass (kg)

The vehicle velocity is obtained by integrating the acceleration signal:

$$v(t) = \int a(t) dt$$

The AEB system automatically reduces vehicle speed when a potential collision is detected, thereby improving road safety and reducing the risk of accidents.

10.4 Block Diagram

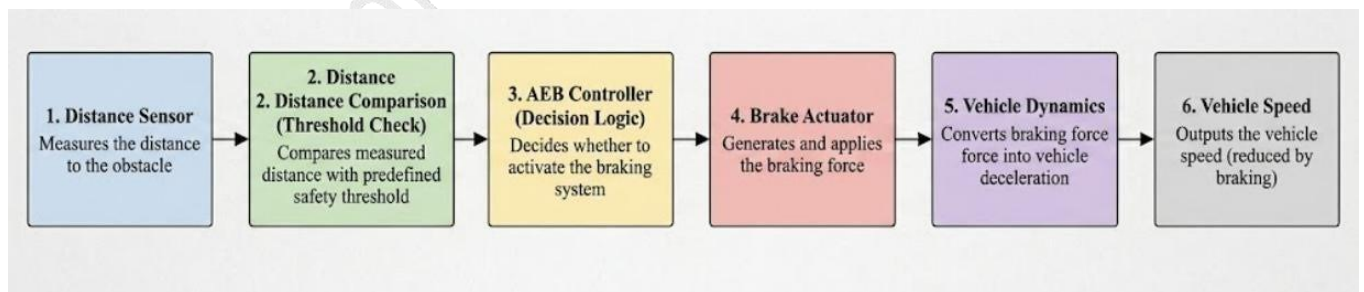


Figure 10.1 Block diagram of the ADAS with Automatic Emergency Braking system.

10.5 Procedure

1. Open MATLAB and create a new Simulink model.
2. Add the Ramp, Constant, Relational Operator, Switch, Gain, Integrator, Saturation, Mux, and Scope blocks.
3. Configure the Ramp block to represent the decreasing obstacle distance.
4. Set the safety threshold distance to 10 m.
5. Configure the Relational Operator block to compare the obstacle distance with the threshold value.
6. Configure the Switch block to activate braking when the distance becomes smaller than the threshold.
7. Set the braking force and vehicle mass parameters using Gain blocks.
8. Configure the Integrator block with an initial vehicle speed of 20 m/s.
9. Add a Saturation block to prevent negative vehicle speed.
10. Connect all blocks according to the ADAS-AEB architecture.
11. Run the simulation and observe the vehicle speed response.

10.6 Simulink Model

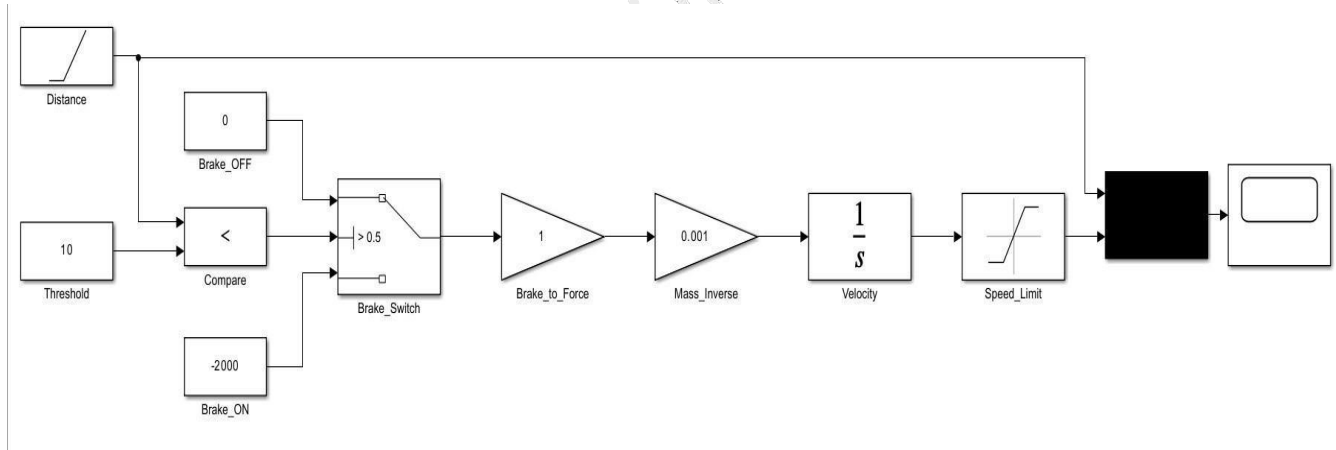
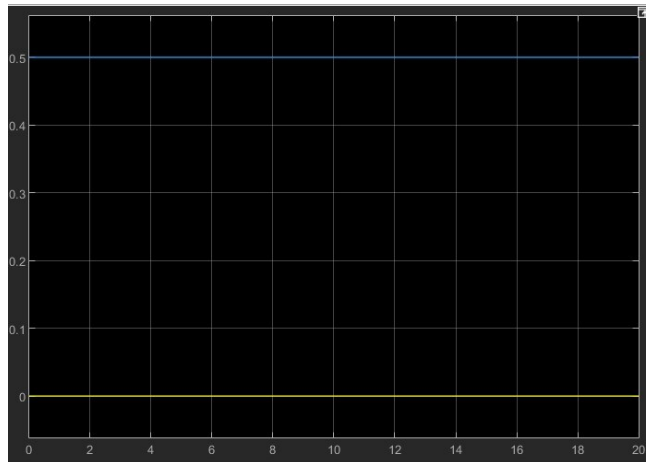
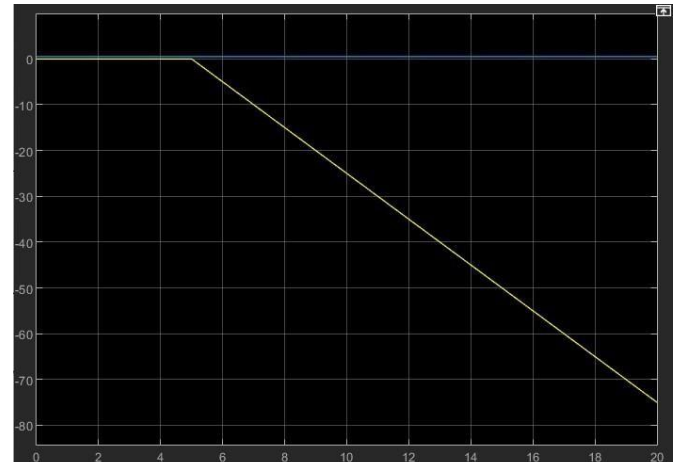


Figure 10.2 Simulink model of the ADAS-AEB system. 10.7

Simulation Results



Brake Command = OFF, if Distance $\geq$ Threshold



Brake Command = ON, if Distance < Threshold

Figure 10.3 Simulation response of the ADAS-AEB system.

The simulation shows that the obstacle distance decreases continuously with time. When the distance becomes less than the predefined threshold of 10 m, the AEB controller automatically activates the braking system. The applied braking force generates vehicle deceleration, causing the vehicle speed to decrease. The speed eventually approaches zero, indicating successful emergency braking.

10.8 Observations

- The obstacle distance decreases continuously during the simulation.
- The AEB controller continuously monitors the distance between the vehicle and the obstacle.
- Braking is activated automatically when the distance falls below the safety threshold.
- The braking force generates a deceleration that reduces vehicle speed.
- The saturation block prevents the vehicle speed from becoming negative.
- The system successfully demonstrates automatic emergency braking functionality.

10.9 Result

An Advanced Driver Assistance System (ADAS) with Automatic Emergency Braking (AEB) was successfully modeled and simulated using MATLAB/Simulink. The system accurately detected a critical obstacle distance and automatically applied braking to reduce vehicle speed and prevent a potential collision.

10.10 Conclusion

The ADAS-AEB system was successfully implemented and analyzed using MATLAB/Simulink. The simulation demonstrated the operation of automatic emergency braking based on obstacle detection and safety distance monitoring. The system effectively reduced vehicle speed when the obstacle distance became critical, thereby improving vehicle safety and collision avoidance performance.